香山昆明湖 V2

自定义向量点积指令

第二阶段：指令设计与基础测试

设计、验证与性能评估

三阶段工程技术报告

2026 年 11 月 8 日修订

目录

第二阶段：指令设计与基础测试 **1**

2.1 设计目标与实施范围 **1**

2.2 指令语义与编码 **1**

2.2.1 八项有符号点积与结果布局 1

2.2.2 数值范围与中间位宽 2

2.2.3 编码与合法执行状态 3

2.3 数据通路与弹性两槽控制 **4**

2.3.1 乘积槽与结果槽 4

2.3.2 时钟、复位与连续流动 4

2.3.3 背压、重定向与事务守恒 6

2.4 RTL 实现与处理器接入 **8**

2.4.1 关键实现语句 8

2.4.2 译码、配置依赖与写回 9

2.4.3 共享 VFEX4 的派发与门控 9

2.5 基础功能与分层验证 **10**

2.5.1 正数点积测试 11

2.5.2 负数点积测试 12

2.5.3 边界值点积测试 12

2.5.4 算术数据通路测试汇总 12

2.5.5 响应配对与复查位置 13

2.6 设计取舍与扩展 **15**

2.6.1 两槽结构与结果保存 15

2.6.2 十六项 INT8 点积的功能进展 15

2.6.3 元素位宽的参数化验证 15

2.7 源码与证据入口 **16**

第二阶段：指令设计与基础测试

2.1 设计目标与实施范围

点积把两组数据逐项相乘，再将乘积求和，是量化矩阵运算的重要计算步骤。本设计在昆明湖 V2 已有向量路径中增加八项 INT8 点积指令，将乘法与归约组合为专用数据通路，同时复用处理器的译码、寄存器重命名、调度、写回和精确提交机制。

本次提交和评估的是弹性两槽实现：乘积槽 P 与结果槽 R 组成两级流水，接收请求后最早 2 拍给出结果，每拍可以接收一笔新请求。工程同时保留更早的单请求实现，可在构建时切换，用于对照和回退。本章依次介绍指令语义、两槽数据通路与控制、处理器接入和三类基础测试，分别对应赛题 2.1 的设计说明、2.2 的 RTL 实现和 2.3 的三类基础测试；十六项点积和 INT16 参数化两项扩展列在本章末尾。主要结果是：指令按赛题语义实现，正数、负数和边界值三类测试的完整 128 位结果均与期望一致；两槽实现达到每拍发起一笔、最早 2 拍响应。

图2-1给出 vdot 在昆明湖 V2 流水线中的位置。vdot 与普通向量指令一样经过取指、译码、重命名和派发；译码器按 custom-0 编码识别它，派发到向量执行单元 VFEX4 中新增的 VDot 功能单元，结果经共享写回送入向量寄存器堆，最后由 ROB 按程序顺序提交。新增和修改的部分只有译码识别、VDot 功能单元和 VFEX4 的派发与写回选择，其余流水线机制全部复用。

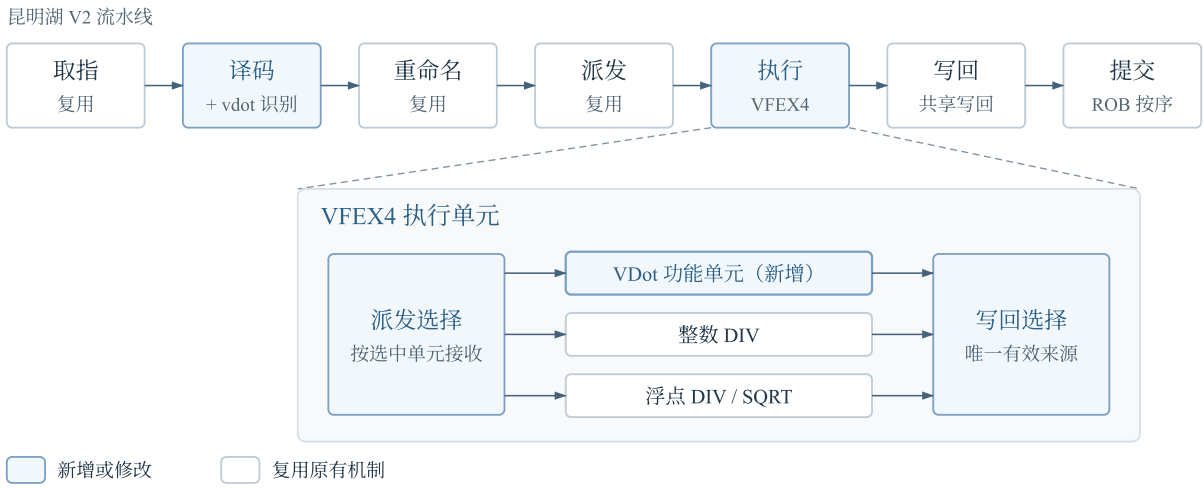


图 2-1 vdot 在昆明湖 V2 流水线中的位置

表2-1汇总了本设计的几项主要决策。

2.2 指令语义与编码

2.2.1 八项有符号点积与结果布局

本指令采用 128 位向量寄存器（VLEN=128）、8 位元素（SEW=8）和单寄存器分组（LMUL=1）；有效元素数为 8（VL=8），起始执行索引为 0（vstart=0）。指令读取两个源寄存器的低 8 个字节，

表 2-1 主要设计决策

决策	做法	理由与效果
指令编码	custom-0 编码，译码为单个微操作	使用 RISC-V 为自定义扩展保留的编码空间，不占用标准 RVV 编码；复用现有译码、重命名和精确异常机制
接入位置	放入已有的向量执行单元 VFEX4，与整数 DIV、浮点 DIV/SQRT 共享派发和写回	不新增发射端口和写回端口；需要处理与 DIV 的互斥、取消和时钟门控（见第2.4.3节）
流水结构	弹性两槽：乘积槽 P + 结果槽 R	相邻请求的乘法和归约重叠执行，发起间隔从上一版单笔在途设计的 2 拍缩短到 1 拍；独立批量周期减少 39%（见第三阶段）
结果保存	R 槽只保存 32 位，输出时补 96 位零	利用结果高 96 位恒为零的性质减少寄存器，写回接口仍是完整的 128 位
运算拍数	乘法和归约分两拍完成	另实现并验收了一拍方案 SP1；两拍方案面积更小、VDot 路径时序余量更大，综合权衡后采用（见第2.6.1节）
DIV 取消修复	DIV 请求被取消后不再保持时钟	取消后 VFEX4 可以完全关钟，测量周期不变

将每个字节解释为有符号整数，计算

$$S = \sum_{i=0}^{7} \mathrm{int8}(vs1[8i+7:8i])\,\mathrm{int8}(vs2[8i+7:8i]).$$

目的低 32 位保存 S 的二补码，高 96 位写零。运算从零开始累加，读取指令执行前的源值，因此支持目的与源寄存器重合，也支持使用 v0 作为普通源或目的。每个源以 128 位向量传入数据通路，八项算术使用其中低 64 位的 8 个元素。

图2-2汇总了指令字段、源操作数取位和结果布局。custom-0 编码给出两个源寄存器和一个目的寄存器；两源各取低 64 位的 8 个有符号字节，同编号字节两两相乘后求和；目的低 32 位保存和的补码，高 96 位写零。以 $S = -8$ 为例，完整结果为 0x00000000_00000000_00000000_FFFFFFF8。

2.2.2 数值范围与中间位宽

有符号 8 位数的范围为 -128 至 127，单个乘积范围为 $[-16256, 16384]$，八项和范围为 $[-130048, 131072]$。乘积使用 16 位；按二项、四项、八项归并时，精确位宽依次增加为 17、18、19 位。正端点 131072 比 18 位有符号数的最大值大 1，因此极值用例能直接检验归约过程中是否保留了完整进位。

归约结果先符号扩展为 32 位，再与 96 位零拼接。负数的符号由低 32 位字段表达，例如 -8 的低 32 位为 `0xfffffff8`，完整 128 位结果仍以 96 个零作为高位。也就是说，19 位用于保证求和精确，32 位是结果槽实际保存的宽度，128 位是写回向量寄存器的架构宽度。



图 2-2 八项点积指令字段与完整结果布局

2.2.3 编码与合法执行状态

指令采用 custom-0 编码，`MATCH=0x0200000b`、`MASK=0xfe00707f`，各字段取值见表2-2。汇编记法为 `vdot.vv vd, vs2, vs1`。

表 2-2 vdot 指令的编码字段

字段	位段	含义或取值
funct6	31:26	`000000`
vm	25	`1`
vs2、vs1	24:20、19:15	两个向量源寄存器编号
funct3	14:12	`000`
vd	11:7	向量目的寄存器编号
opcode	6:0	custom-0，`0x0b`

例如 `vdot.vv v10,v8,v9` 对应 `0x0284850b`。标准汇编器不认识这条自定义指令，程序通过经过校验的编码宏或直接写入机器字来生成它。

向量功能启用、vill=0、vm=1 以及本节的 SEW、LMUL、VL 和 vstart 共同确定合法状态，四种 vta/vma 策略组合均适用。合法执行完成一次完整目的写回，并保持 VL、vtype、vlenb、vxrm、vxsat 和 vcsr。非法请求携带 illegal-instruction 异常完成，同时关闭向量与 v0 目的写使能，由处理器的精确异常机制处理。

2.3 数据通路与弹性两槽控制

2.3.1 乘积槽与结果槽

点积数据通路包含 8 个有符号 8 × 8 乘法器、乘积槽 P、归约网络和结果槽 R。P 保存 8 个 16 位乘积及这笔请求的控制、性能信息；R 保存 32 位结果及同一请求的身份。结果从 R 输出时再补入 96 位零，这样 R 只需保存 32 位，写回接口仍是完整的 128 位。

图2-3给出这条数据通路的结构。8 个乘法器的 16 位乘积存入 P，组合归约网络按 4 个 17 位、2 个 18 位、1 个 19 位逐级相加得到精确和，符号扩展为 32 位后存入 R，输出时再与 96 位零拼接成 128 位结果。图下部为事务身份通路：ROB、目的寄存器、写使能、异常信息和性能字段存入 pCtrl 与 pPerf，P 转入 R 时整包移入 rPacket；若 P 中已锁存非法指令标记，选择器以零代替结果，并关闭目的写使能。

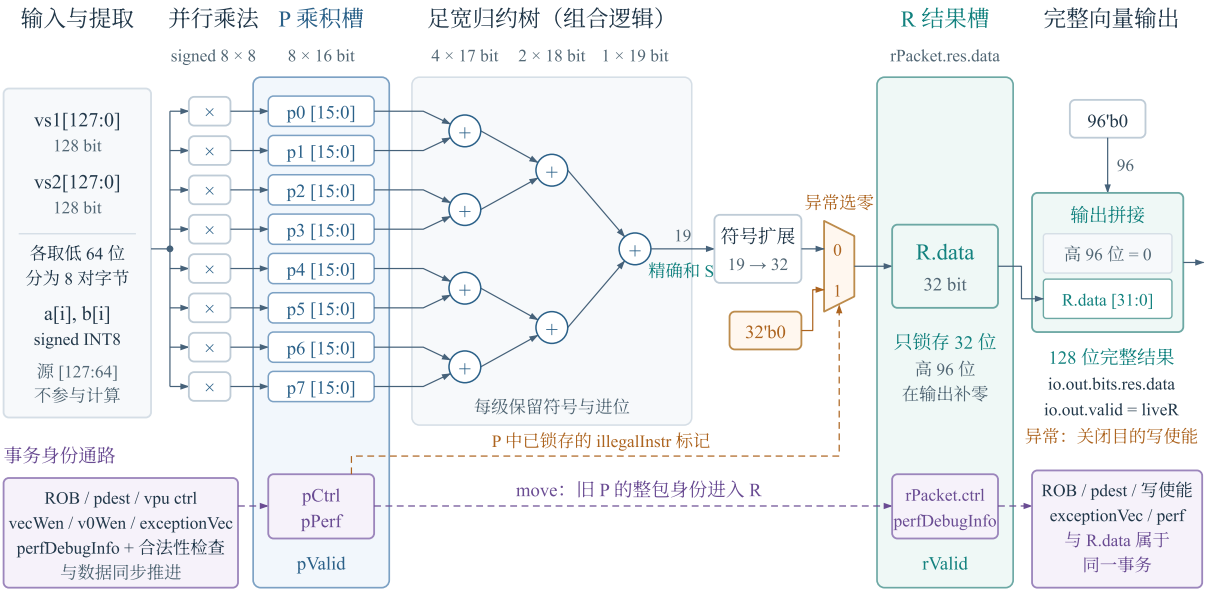


图 2-3 八项点积的乘积槽、归约网络和结果槽

两槽的有效位表示各自是否保存一笔请求。重排序缓冲区（ROB）编号、目的物理寄存器、向量或 v0 写使能、异常信息以及性能字段与数据同时移动，使写回始终能够找到正确的目的。输出停顿时，R 中已有的请求保持不变，不受新输入影响。

2.3.2 时钟、复位与连续流动

VDot 功能单元（FU）接入共享执行单元 VFEX4，后文将 VFEX4 称为父级。实现沿用功能单元所在的单时钟域和父级复位连接。P、R 在时钟上升沿更新；复位清除两槽有效位，并抑制复位期间的输入和输出握手。在父级选择并允许接收 VDot、独立请求连续供给、下游持续就绪且没有复位或重定向时，请求于 E0 进入 P，E1 完成归约并进入 R，E2 是最早的输出握手沿。连续请求的发起间隔为 1 拍。表2-3列出 P、R 与下游状态的典型组合及对应的本拍动作。

图2-4以四笔连续请求为例展示逐拍流动，图中 P、R 两行为时钟沿前的状态。下游持续就绪时，#1 在 cycle 10 被接收，cycle 11 位于 P，cycle 12 位于 R 并完成响应；在 cycle 12 这一拍，接收 #3、#2 由 P 迁入 R、#1 从 R 响应三件事同时完成。由此得到 1 拍的发起间隔和接收后 2 拍的最早响应。

表 2-3　两槽状态与本拍动作

P 状态	R 状态	下游状态	本拍动作
空	空	任意	接受新请求到 P
有效	空	任意	原 P 进入 R，同时可接收下一请求
空	有效	反压	保持 R，P 可接收一笔请求
有效	有效	反压	保持两槽，暂停接收
有效	有效	就绪	原 R 输出、原 P 进入 R、新请求进入 P

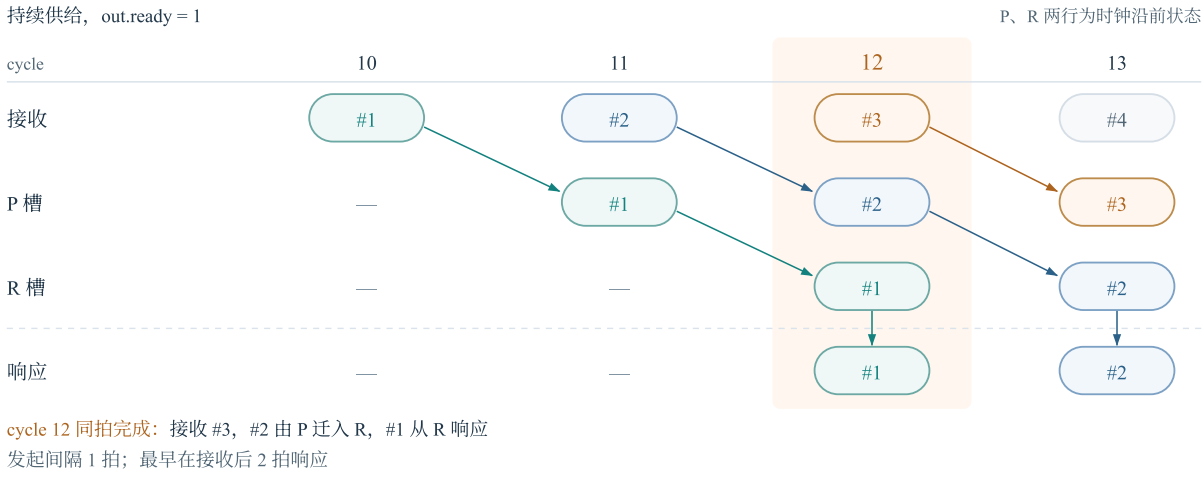


图 2-4　连续请求在两槽中的逐拍流动

全核运行的观察器日志也记录到了这种同拍重叠，如图2-5所示：10986 周期同时记录了 ACCEPT seq2、MOVE seq1 和 RESPONSE seq0，分别对应新请求进入 P、原 P 推进至 R 和原 R 响应。

全核运行中两槽观察器的日志摘录

```
line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
```

图 2-5　连续事务的同拍重叠记录

FU 能否接收取决于自身是否有空槽，但新请求何时派发仍由父级根据当前选择和共享资源

占用决定。

时延指一笔请求从被接受到产生响应所用的拍数，发起间隔指相邻两笔请求被接受的间隔。两槽结构支持连续独立点积；程序的最终周期还由数据依赖、调度、访存和写回共同决定。FU 通过 valid/ready 报告完成，下游反压时响应相应顺延。

2.3.3 背压、重定向与事务守恒

R 有效而下游未就绪时，数据、目的和控制包一起保持。P 可利用空槽再接收一笔请求；若两槽均占用且 R 仍受阻，输入 ready 撤销，直到 R 输出或取消腾出空间。

图2-6给出一次实际背压过程，A、B 分别为序号 1321 和 1322 的请求。A 在 +2 拍进入 R，下游从 +2 到 +8 连续 7 拍未就绪，期间 R 保持 A 的结果与目的身份，P 中的 B 也保持不变；+9 拍下游恢复就绪，A 完成一次响应，B 同拍由 P 迁入 R。此时 FU 已经腾出空槽（FU 的 in.ready 为 1），但父级的 in.ready 仍为 0：FU 有空槽并不等于立即接收新请求，何时派发仍由父级决定。

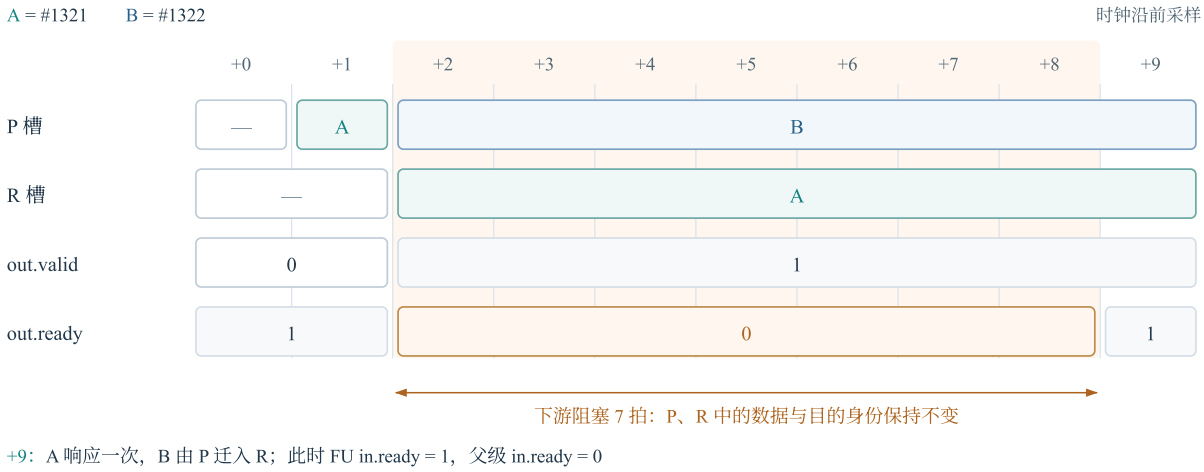


图 2-6 背压期间的数据与目的身份保持

重定向按两个槽各自的 ROB 年龄独立判断取消，保留较老的幸存请求，并在该拍暂停新请求接收。被取消的 R 停止输出，被取消的 P 释放待归约状态；复位的优先级最高。ROB 环回场景使用完整指针年龄关系确定先后。

图2-7给出四种取消与复位场景，均假设下游未就绪（out.ready=0），且重定向当拍不接收新请求。只取消 P 时，R 中的 B 保留，继续等待下游就绪；只取消 R 时，P 中的 A 推进到 R；两槽同时被取消时都变为无效；reset 与 redirect 同拍时以 reset 为准，两槽清空。

每次验证在统一时钟坐标下检查

$$N_{\text{accept}} = N_{\text{response}} + N_{\text{cancel}} + N_{\text{reset}} + N_{\text{inflight}}, \qquad 0 \le N_{\text{inflight}} \le 2.$$

该关系将每笔已接受请求归入响应、取消、复位丢弃或当前占用之一，并在运行结束时检查两槽排空。每笔响应还核对完整 128 位结果、ROB 和目的身份。

图2-8综合了正常推进与取消、复位处理。A 部分以 (P.valid, R.valid) 表示状态，标出没有 flush 和 reset 时各状态之间的转移及触发事件；B 部分给出取消与复位的判定顺序：reset 优先清零两槽，未复位时按各自 ROB 匹配取消，幸存请求仍可移动或输出，flush 有效期间禁止新请求进入；C 部分说明接收、移动和输出三件事可以在同一边沿重叠。

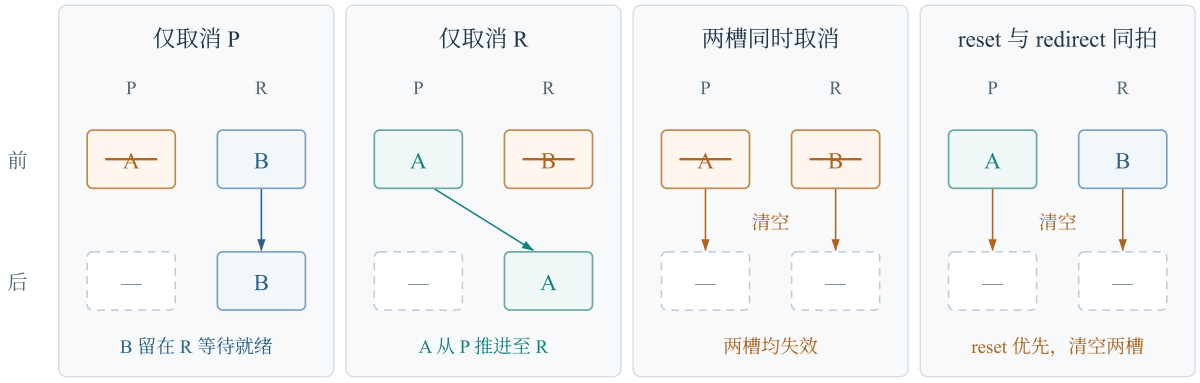


图 2-7　部分取消、全部取消与复位恢复

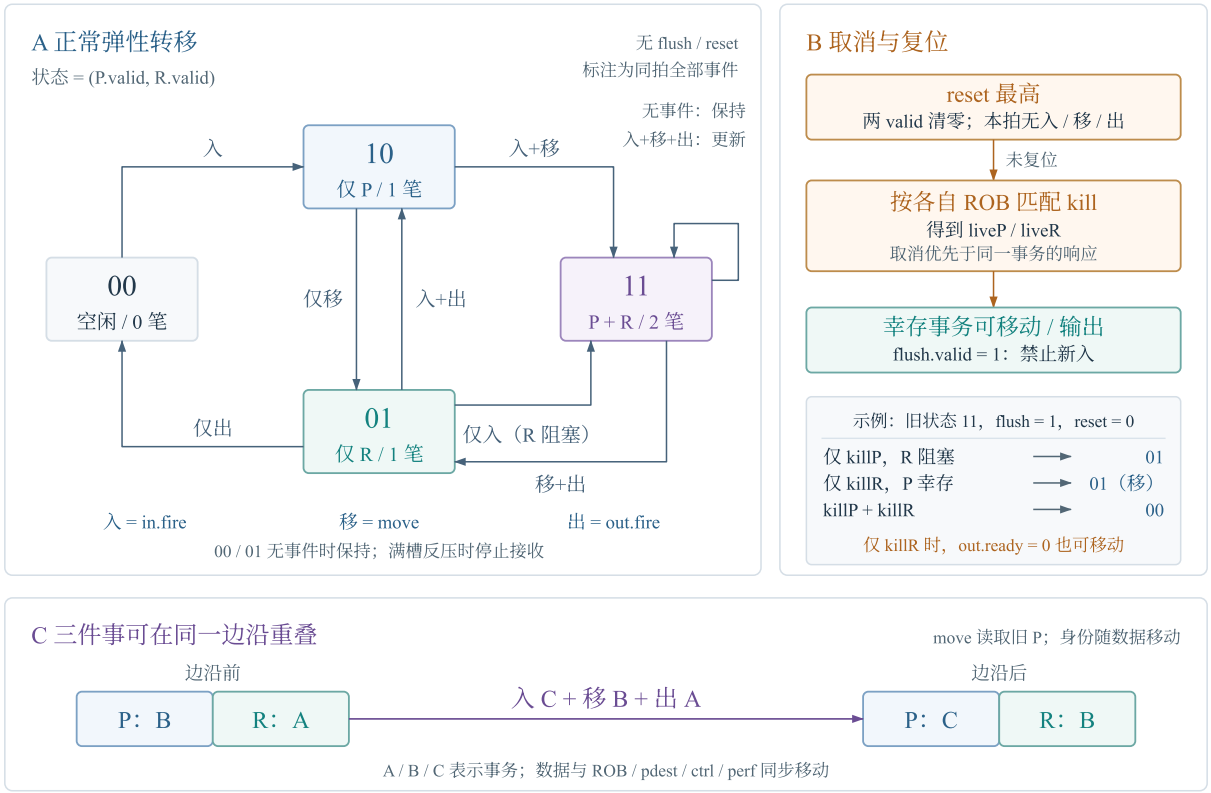


图 2-8　两槽同拍推进与取消、复位处理

2.4 RTL 实现与处理器接入

2.4.1 关键实现语句

算术部分位于 src/main/scala/xiangshan/backend/fu/vector/VDotDatapath.scala，乘积和每级归约按有符号类型计算，并逐级保留进位，对应的源码摘录见图2-9。wrapper/VDotElastic.scala 保存 P、R 的控制和数据，图2-10依次摘录了其中的 32 位结果槽寄存器、握手与取消条件，以及状态更新和原 P 内容转入 R 的逻辑。

（a）受测版本的 VDotProducts 源码

```
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

（b）受测版本的 VDotReduce 源码

```
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

图 2-9 八项点积算术实现摘录

下面摘示从 P 转入 R 的关键语句：

```
when(move) {
  rPacket.ctrl := pCtrl
  rPacket.perfDebugInfo := pPerf
  rPacket.res.data :=
  Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
    0.U(32.W), reduce.io.result(31, 0))
}
```

这些赋值读取本拍边沿前的 P 内容，因此同拍接收的新请求会进入 P，原请求的结果和身份则进入 R。公开输出使用：

```
io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

结果寄存器的保存位宽为 32 位，高 96 位由输出组合逻辑补零。

（a）槽寄存器与 32 位结果状态

```
    require(!cfg.writeFFlags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

（b）握手与取消条件

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

（c）状态更新与原 P 内容转入 R

```
PS C:\WINDOWS\system32>
                       > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

图 2-10　两槽状态与控制实现摘录

2.4.2　译码、配置依赖与写回

精确编码匹配后，译码器将 vdot 归入向量运算，生成单个微操作并建立两个向量源依赖。实际 VL 通过重命名后的依赖传递，SEW、LMUL 和其他向量控制随指令保存。这样，多条向量配置与运算交错时，每条点积仍使用与自己对应的配置。

旧目的值不参与累加，译码去除相应的旧目的依赖；向量和 v0 的目的分配、异常信息及最终写回沿用现有处理器机制。两槽实现通过构建开关 `VDOT_O2_ELASTIC=1` 和 `VDOT_T14_FUSED_INPUT=1` 启用；不加开关时构建单请求实现，便于对照和回退。

2.4.3　共享 VFEX4 的派发与门控

VDot 与整数 DIV、浮点 DIV/SQRT 共用 VFEX4 父级。父级根据选择、实际占用和被选功能单元的 ready 进行原子派发，使父级和子单元接受的是同一笔请求。VDot 两槽排空后可以切换到 DIV；DIV 在途期间保存自己的 ROB 身份和占用状态。输出按唯一有效完成来源选择共享写回。

图2-11是这一共享结构的简化示意。派发与互斥逻辑按选中的 FU 接收请求，三个功能单元各有独立的时钟门控，完成结果经共享写回从唯一有效来源输出；占用与控制逻辑汇总各单元的状态，约束派发，并产生取消和时钟使能。图中省略了部分逐信号连线。

派发许可同时检查资源占用、请求选择以及 flush 和 reset。上游 ready 由许可与被选 FU 的

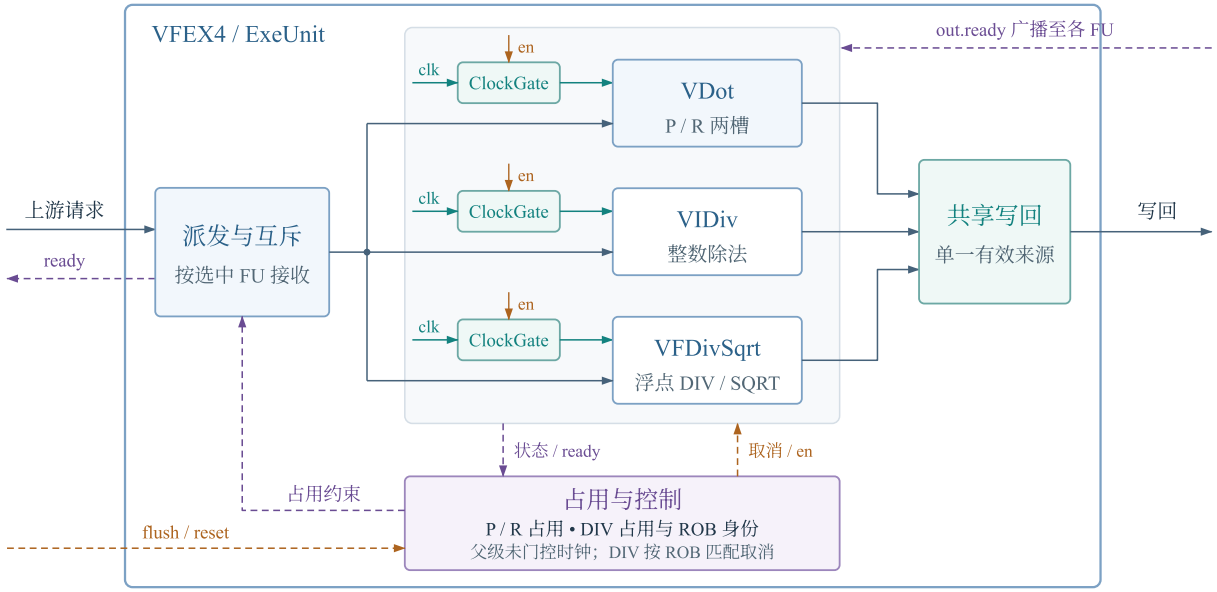


图 2-11 VFEX4 中的派发、资源互斥与共享写回

ready 共同决定，输入 valid 也仅送往被选单元，因此父级接受与该 FU 接受必须发生在同一边沿。ready 的返回路径按选择信号取值，不能将各 FU 的 ready 直接作或运算。

VDot 占用由 `pValid` 或 `rValid` 表示；DIV 侧还保留在途 ROB 和占用状态，接受后持续保持，直到输出握手或匹配取消。图2-11中的输出 ready 向各 FU 广播，但输出 valid 最多一路有效，共享写回按该唯一有效来源同时选择结果与身份。

时钟使能覆盖存活请求、当前输入接受和必要的取消、复位清理。对应验证包含满槽背压、部分取消、ROB 环回、关钟后的复位恢复，以及 VDot 和两类 DIV 之间的切换。DIV 取消修复让被取消的 DIV 请求不再占住时钟，父级因此能进入空闲并接受后续工作。

图2-12给出两段实测过程。上半部分是 VDot 排空后切换到 DIV：旧 R 输出、VDot 排空和 DIV 接收依次相隔 1 拍，整数 DIV 与浮点 DIV 从接收到响应分别用 10 拍和 8 拍。下半部分是取消清理后的时钟变化：DIV 在 +0 拍接收、+1 拍完成取消清理，+2 至 +4 拍父级全部关钟，+5 拍重新接收 VDot，+7 拍给出响应。也就是说，取消清理完成后 DIV 不再保持时钟，父级可以完全关钟，之后再正常切换回 VDot。

2.5 基础功能与分层验证

验证首先用独立数学计算确定完整 128 位结果，再依次检查算术 RTL、带握手的功能单元、真实父级和全核执行。算术测试关注有符号乘法、极值和逐级位宽；FU 与父级测试加入背压、取消、复位、目的身份和 DIV 交互；整核协同验证进一步检查译码、架构状态及提交结果。

图2-13按测试输入、五层验证和期望与检查三列组织这些测试。算术测试输入同时驱动独立数学模型和算术 RTL，前者给出完整 128 位期望，与后者的实际值比较；分层事务用例分别驱动 VDot FU 和真实 VFEX4 父级，检查反压、取消、复位、身份，以及父级的互斥、切换和空闲关钟；AM 功能矩阵在全核 EMU 与 NEMU 上运行，通过 DiffTest 和程序自检核对架构状态。算术 RTL 的 10022 项输入与全核 117 个独立用例、124 次执行分别统计，各层证据独立归档。

以下三类测试对应赛题 2.3.1 至 2.3.3。图2-14至图2-16的输入与响应取自固定八项两槽版本的

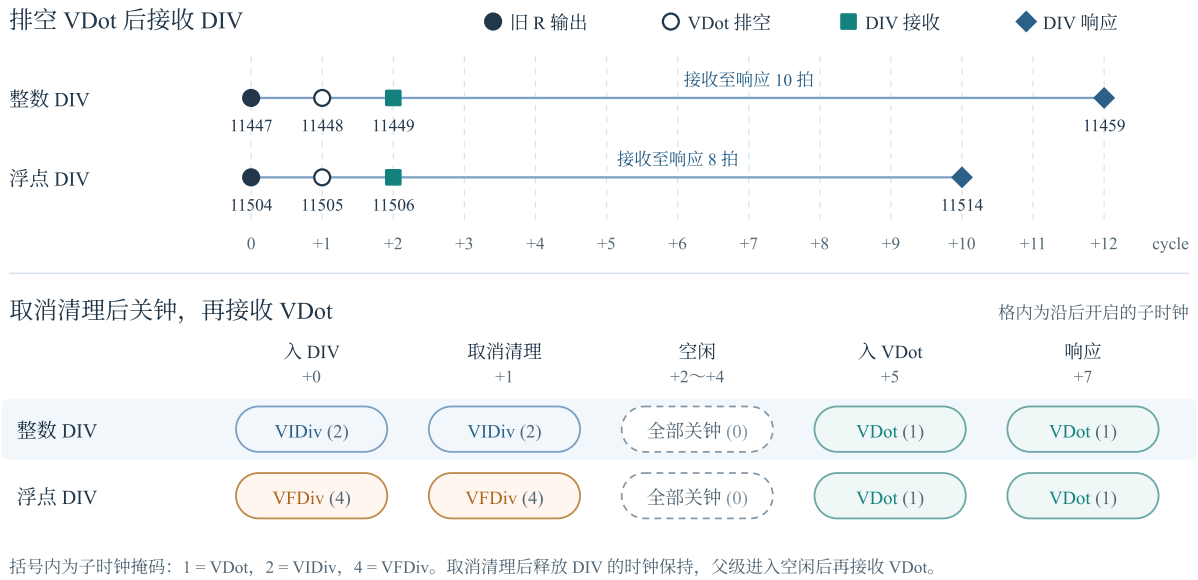


图 2-12　共享资源切换、取消与时钟保持

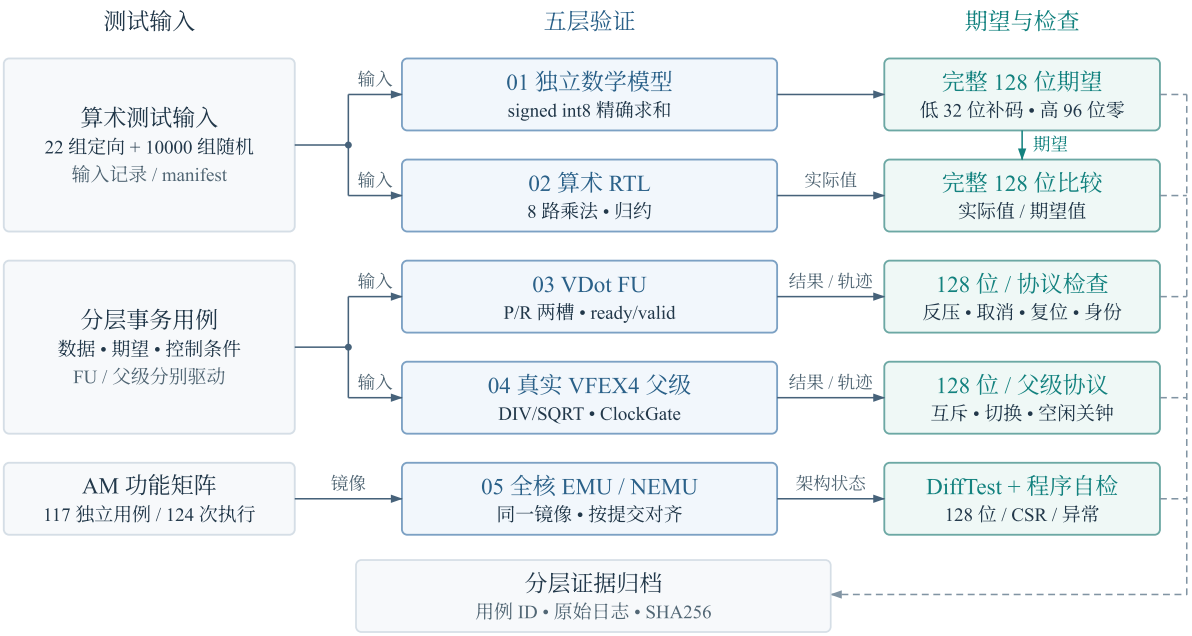


图 2-13　从数学结果到整核执行的验证层次

全核观察记录，按 seq、ROB 和目的物理寄存器配对；第三阶段说明该版本的架构差分验证。

2.5.1　正数点积测试

两组有效输入均为 $[1, 2, 3, 4, 5, 6, 7, 8]$，八项平方和为 204。全核功能测试程序 functional-2 的运行记录中，实际完整 128 位结果为 0x000000000000000000000000000000cc，与预期一致。源高 64 位填入 A5 哨兵，用于检查硬件是否准确选择了低 8 个元素。图2-14摘录了这次响应的原始日志字段，除源值、预期与实际结果外，还记录了请求序号 seq=1、ROB=0:39（环回标志 0、编号 39）、目的物理寄存器 pdest=103，以及接受和响应周期。

```
[POSITIVE] functional-2  seq=1  ROB=0:39  pdest=103
vs1 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
expected sum      204
expected (128-bit)  0x000000000000000000000000000000cc
actual   (128-bit)  0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
source: clean-02/fullcore-run-01/runs/functional-2/run.log
SHA256: 90b36b19e9d141e1b0bdad6bed461b4dbde4e534798f5fbdaa8819d9e2ef3211
```

图 2-14　正数点积：预期与实际结果均为 204

2.5.2　负数点积测试

有效输入为八个 −1 与八个 1，预期点积为 −8。`functional-0` 记录的完整结果为 0x0000 0000000000000000000ffffff8。低 32 位以二补码表达负数，高 96 位保持零，符合指令规定的结果布局。对应的日志摘录见图2-15，该响应记录为 seq=4、ROB=0:12、pdest=143。

```
[NEGATIVE] functional-0  seq=4  ROB=0:12  pdest=143
vs1 (128-bit)     0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)     0xa5a5a5a5a5a5a5a501010101010101
expected sum      -8
expected (128-bit)  0x0000000000000000000000fffffff8
actual   (128-bit)  0x0000000000000000000000fffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-15　负数点积：低 32 位保存负数补码

2.5.3　边界值点积测试

最大正结果由八个 −128 与八个 −128 产生。每项乘积为 16384，总和为 131072；`functional-0` 记录的完整结果为 0x00000000000000000000000000020000。这一输入检验了乘法和归约在正端点的精确性。对应的日志摘录见图2-16，该响应记录为 seq=5、ROB=0:43、pdest=66。

零输入测试采用八个 0 与八个 127，实际完整 128 位响应为零。负端点由八个 −128 与八个 127 构成，预期总和为 −130048，完整结果的低 32 位为 `0xfffe0400`、高 96 位为零；这一用例属于算术 RTL 的 22 项定向测试，已随定向测试全部通过。

2.5.4　算术数据通路测试汇总

算术 RTL 测试包含 22 项定向输入和 10000 项随机输入，共 10022 项全部通过。定向输入针对符号、零值、极值和元素选择，随机输入进一步扩大数值组合覆盖。图2-17为测试程序输出的三行 PASS 汇总，描述的是算术数据通路；带背压和取消的时序行为在 FU 及父级测试中检查。

```
[BOUNDARY] functional-0  seq=5  ROB=0:43  pdest=66
vs1 (128-bit)     0xa5a5a5a5a5a5a5a580808080808080808080
vs2 (128-bit)     0xa5a5a5a5a5a5a5a580808080808080808080
expected sum      131072
expected (128-bit)  0x00000000000000000000000000020000
actual   (128-bit)  0x00000000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
source: clean-02/fullcore-run-01/runs/functional-0/run.log
SHA256: 5bd56636056d7b8c77ab976511522c6654c112644f0ea024dfac99bb92f855f7
```

图 2-16　正端点：八对最小 INT8 数产生 131072

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

图 2-17　算术 RTL 的定向与随机测试汇总

2.5.5　响应配对与复查位置

四类已展示响应均按 seq、ROB 和 pdest 与接受记录配对，并比较完整 128 位结果。具体位置见表2-4，可与前述日志摘录中的输入和响应互相核对。零输入对应 seq=0、ROB=0:8、pdest=124。

表 2-4　已展示响应的日志位置

输入类型	日志	接受/响应周期	原始行号
正数	`functional-2/run.log`	339296 / 339298	7030–7034
负数	`functional-0/run.log`	420291 / 420293	7927–7931
最大正结果	`functional-0/run.log`	448417 / 448419	8400–8404
零输入	`functional-0/run.log`	311084 / 311086	5814–5818

原始记录保存在受测版本干净环境复现的运行归档（clean-02/fullcore-run-01/runs/）中；本节展示执行单元响应，完整运行的架构差分结果见第三阶段。

图2-18把上述三类测试放在一起对照。三组输入的源高 64 位都填入 A5 哨兵，不参与运算；三个结果分别为 204、−8 和 131072，写回的高 96 位均为零，与期望一致。底部的 10022 项统计与图2-17是同一组算术测试。

算术 RTL 之外，团队还用 Python 在主机上实现了一个独立的参考模型，用来定义指令语义并生成测试向量；它的 35 个单元测试也全部通过，输出见图2-19。这里按测试方法计数，与算术 RTL 的 10022 组输入分别统计。

在基础数值之外，测试还检查源高半区哨兵、旧目的哨兵、寄存器别名、v0 目的、非法配置和向量状态恢复。在 FU 和真实父级上，还向响应身份、写使能、取消状态和结果高位注入错误，确认检查器能够发现；正常运行时则核对完整响应和两槽排空。



图 2-18　正数、负数与边界测试的输入和结果

主机参考模型的单元测试输出

```
HELLO_VADD_EMU_RUN_PASS
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- bash /home/riscv/riscv-lab/zjhwork2-review-20261018/repo/team/scripts/check-vdot-model.sh
test_check_detects_corruption_and_leaves_every_file_unchanged (test_generate.GeneratorTests) ... ok
test_check_missing_output_does_not_create_directory (test_generate.GeneratorTests) ... ok
test_check_without_python_B_is_readonly_and_detects_source_drift (test_generate.GeneratorTests) ... ok
test_directed_manual_expectations_and_formats (test_generate.GeneratorTests) ... ok
test_manifest_hashes_and_smoke_selection (test_generate.GeneratorTests) ... ok
test_prng_contract_byte_order_and_repeatability (test_generate.GeneratorTests) ... ok
test_state_aliases_use_one_real_prestate (test_generate.GeneratorTests) ... ok
test_10000_fixed_seed_pairs_with_independent_byte_oracle (test_model.ArithmeticTests) ... ok
test_D00_through_D10 (test_model.ArithmeticTests) ... ok
test_all_65536_single_signed_byte_products (test_model.ArithmeticTests) ... ok
test_contract_identity_and_dimensions (test_model.ArithmeticTests) ... ok
test_extrema_require_19_signed_bits (test_model.ArithmeticTests) ... ok
test_high_half_is_ignored_and_negative_is_not_128bit_sign_extended (test_model.ArithmeticTests) ... ok
test_lane_order_and_one_hot_inputs (test_model.ArithmeticTests) ... ok
test_all_32768_register_encodings (test_model.EncodingTests) ... ok
test_every_fixed_bit_flip_is_illegal (test_model.EncodingTests) ... ok
test_known_word_and_source_field_order (test_model.EncodingTests) ... ok
test_random_nonmatching_words_are_illegal (test_model.EncodingTests) ... ok
test_256_random_register_banks_against_byte_oracle (test_model.ExecutionTests) ... ok
test_aliases_and_v0_v31_use_instruction_before_state (test_model.ExecutionTests) ... ok
test_config_is_frozen_and_default_execution_works (test_model.ExecutionTests) ... ok
test_directed_values_with_dirty_old_destination (test_model.ExecutionTests) ... ok
test_illegal_config_has_cause_two_and_no_input_changes (test_model.ExecutionTests) ... ok
test_illegal_encoding_preserves_inputs (test_model.ExecutionTests) ... ok
test_other_vlen_is_unsupported_configuration (test_model.ExecutionTests) ... ok
test_policy_combinations_rounding_saturation_and_vs_dirty (test_model.ExecutionTests) ... ok
test_config_api_types_and_small_field_ranges (test_model.InvalidApiTests) ... ok
test_instruction_input_ranges_and_types (test_model.InvalidApiTests) ... ok
test_lane_inputs_require_exact_length_and_signed_integers (test_model.InvalidApiTests) ... ok
test_packed_sources_and_high_half_require_unsigned_integer_domains (test_model.InvalidApiTests) ... ok
test_register_bank_domains (test_model.InvalidApiTests) ... ok
test_every_serialized_arithmetic_expectation (test_vectors.CommittedVectorTests) ... ok
test_fixed_prng_and_smoke_selection (test_vectors.CommittedVectorTests) ... ok
test_manifest_and_all_file_hashes (test_vectors.CommittedVectorTests) ... ok
test_state_expectations_and_decoder_scope (test_vectors.CommittedVectorTests) ... ok

----------------------------------------------------------------------
Ran 35 tests in 1.353s

OK
OK: 5 files match regenerated bytes in /home/riscv/riscv-lab/zjhwork2-review-20261018/repo/team/models/vdot/data
VDOT_MODEL_CHECK_PASS (host model and data only; no RTL/NEMU run)
PS C:\WINDOWS\system32>
```

图 2-19　独立主机模型的测试输出

2.6 设计取舍与扩展

2.6.1 两槽结构与结果保存

采用两槽结构后，相邻请求的乘法和归约可以重叠执行，连续的独立点积因此能每拍发起一笔。将 R 中的结果保存宽度收窄至 32 位，利用了高 96 位固定为零这一架构性质，减少了数据寄存器成本。每笔请求的 ROB、目的和控制信息则完整保留，以支持乱序执行、背压和精确取消。

运算拍数方面，团队还实现并验收了把乘法和归约合并为一拍的 SP1 方案。它使依赖链周期再降低约 14%，但 FU 面积增加约 6%，VDot 从输入到寄存器的路径时序余量缩小，批量吞吐和函数调用周期基本不变。综合权衡后，最终提交仍采用两拍算术的两槽实现。两版的周期、面积和时序对比见第三阶段。

2.6.2 十六项 INT8 点积的功能进展

在八项实现的基础上，团队增加了独立的十六项点积候选，以补丁形式提供并附完整验证记录。它采用 `MATCH=0x0e00000b`、`MASK=0xfe00707f`，在 e8/m1、VL=16 时读取两个 128 位源的全部 16 个字节。八项指令继续使用原编码和 VL=8 语义。

候选复用两组八项乘积及归约模块。P 保存 16 个 16 位乘积，两组 19 位部分和相加得到 20 位精确和，再扩展为低 32 位结果。十六项和的范围为 $[-260096, 262144]$，R 仍保存 32 位，写回高 96 位为零。接收八项指令时，高八个乘积写零，使后级统一完成归约。两槽握手和取消机制沿用原结构。

2026 年 11 月 6 日，该候选完成了编译开关关闭和开启两种配置下的译码、FU 和真实父级验证。启用十六项扩展后，使用新 EMU 与匹配 NEMU 完成六例整核差分：2 例八项指令、4 例十六项指令全部通过。六例覆盖源高半区贡献、有符号正负端点、选定源目的别名、完整 128 位写回和恢复八项指令。相应 FU 与父级连续请求场景还核对了 2 拍首响应与 1 拍发起间隔。

目前该扩展已完成指令和执行通路的功能验证。它每条处理 16 项，可以减少量化 GEMV 所需的点积指令条数；完整模型调用的性能对比和综合时序分析是下一步工作。本报告的周期和成本数据来自八项受测版本。

2.6.3 元素位宽的参数化验证

另一组独立模块实验把输入元素扩展为 INT16，计算 8 项点积，并使用低 64 位结果字段、高 64 位清零的输出布局。该配置的最大正和为 2^{33}，精确累加至少需要 35 位。参数化乘法与归约模块完成 94 项定向输入和 10000 项随机输入的完整 128 位结果检查，并验证单请求包装中的反压、取消与复位行为。这组实验说明，改变元素位宽时，可以按同样的方法重新确定中间位宽、结果字段和测试向量。

2.7 源码与证据入口

表2-5按内容列出本章涉及的源码与证据入口。各入口下保存完整源码版本、运行输入、原始日志和失败记录，可用于复查实现细节与测量依据。

表 2-5 第二阶段源码与证据入口

内容	阅读入口
指令语义与编码	指令规格与定向向量
算术及两槽实现	八项弹性 FU 源码及乘积与归约源码
构建与版本选择	工程版本入口与可选配置
周期、面积与 STA	八项两槽受测版本的数据与原始报告
十六项扩展	候选结构、补丁与分层验证、六例整核差分结果
元素位宽参数化	八项 INT16 独立模块及完整结果验证